МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена трудового Красного Знамени федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский технический университет связи и информатики»
Кафедра Программная инженерия

Отчет по лабораторной работе №1

“**Основы SQL**”

по дисциплине «Введение в информационные технологии»

Выполнил: студент группы
БПИ2501
Карпеко Никита Александрович
Проверила: Мосева Марина Сергеевна

Москва, 2026

Цель работы

Познакомиться с основными возможностями SQL и научиться выполнять основные возможные задачи.

Содержание для выполнения

1. Создание таблиц

   Для выполнения работы воспользуемся сайтом

   https://sunnygoyal.com/jade/

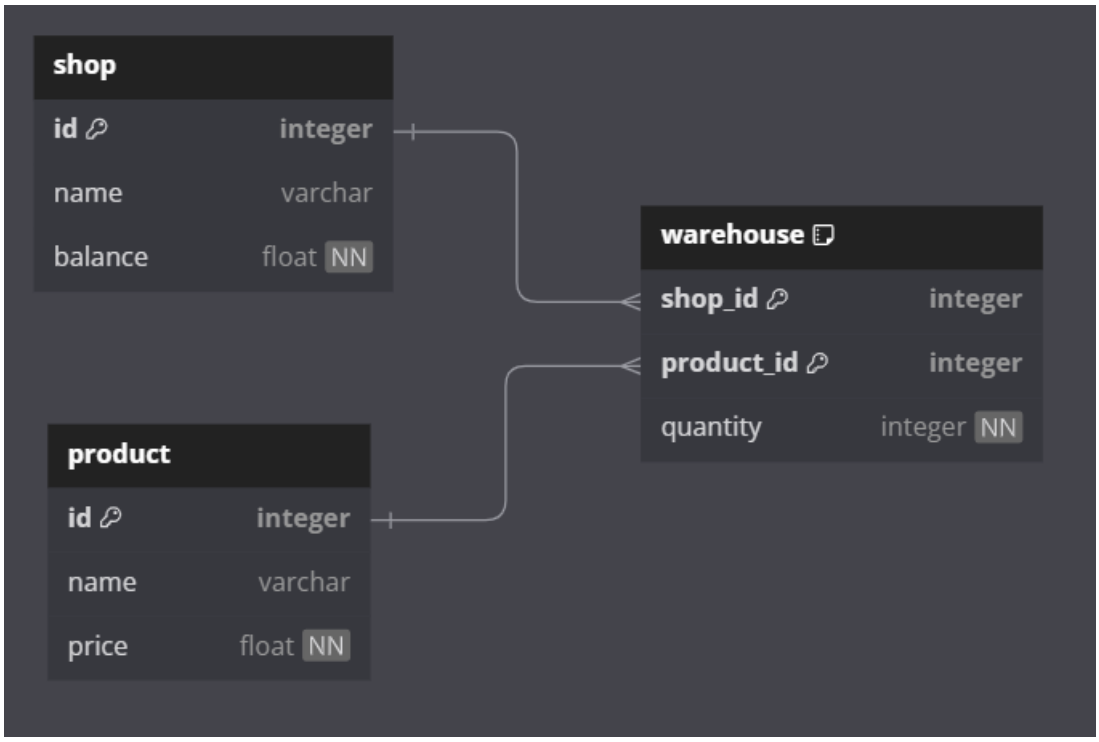


Рисунок 1 – пример схемы базы данных

В качестве примера будем рассматривать создание базы данных для сети магазинов. Создадим таблицы на основе рисунка 1. Для создания требуемых таблиц воспользуемся скриптом ниже.

Листинг 1 – создание таблиц БД

```
CREATE TABLE shop (
id INTEGER PRIMARY KEY,
name VARCHAR(255) UNIQUE,
balance FLOAT NOT NULL);

CREATE TABLE product (
id INTEGER PRIMARY KEY,
name VARCHAR(255) UNIQUE,
price FLOAT NOT NULL);

CREATE TABLE warehouse (
shop_id INTEGER REFERENCES shop (id),
product_id INTEGER REFERENCES product (id),
quantity INTEGER NOT NULL,
PRIMARY KEY (shop_id, product_id));
```

Рассмотрим подробнее листинг 1. Для создания таблицы используется команда CREATE TABLE после чего указывается название таблицы. В скобках производится перечисление полей таблицы. В листинге использовались следующие типы данных INTEGER – целочисленный тип данных, FLOAT – числа с плавающей точкой и VARCHAR – строка переменной длины с указанием предела по количеству символов. Для создания ограничения на незаполненность используется ключевое слово NOT NULL при объявлении переменной. Обозначение первичного ключа осуществляется ключевыми словами PRIMARY KEY в поле, которое должно обладать данным свойством, или в конце отдельной строкой с перечислением названия полей в скобках. Создание внешнего ключа осуществляется при помощи ключевого слова REFERENCES, дальше указывается таблица и поле, на которое идет ссылка. Ключевое слово UNIQUE отвечает за уникальность значения в таблице

2. Внесение данных

Внесем данные в ранее созданные таблицы.

Листинг 2 – внесение данных в таблицу shop

```
INSERT INTO shop (id, name, balance) VALUES (1, 'пятерочка',31);
INSERT INTO shop (id, name, balance) VALUES (2, 'перекресток',133);
```

Листинг 3 – внесение данных в таблицу product

```
INSERT INTO product VALUES (1, 'молоко', 100);
INSERT INTO product VALUES (2, 'хлеб', 25);
INSERT INTO product VALUES (3, 'хлеб', 30);
```

Листинг 4 – внесение данных в таблицу warehouse

```
INSERT INTO warehouse VALUES (1, 1, 20);
INSERT INTO warehouse VALUES (1, 2, 10);
INSERT INTO warehouse VALUES (2, 1, 30);
```

Для внесения данных используются ключевые слова INSERT INTO, далее указывается название таблицы и опционально поля, которые будут заполняться, после ключевого слова VALUES в скобках через запятую идет перечисление значений, которые мы хотим занести в таблицу.

3. Создание запросов

Листинг 5 – Запрос на получение данных таблицы shop

```
SELECT * FROM shop;
```

Для проверки работы выше выполним запрос с листинга 5. Для запросов на получение информации используется слово SELECT, далее указываются требуемые для вывода поля или * в случае, когда необходимо выбрать все поля. После ключевого слова FROM указывается таблица, к которой мы обращаемся.

Листинг 6 – Запрос на получение данных таблицы product.

```
SELECT * FROM product;
```

Пример запроса с указанием полей показан на листинге 6. Отметим, что в результате мы видим всего 2 строки, хотя и вводили 3. Здесь показывается работы ограничения UNIQUE на поле product.name

Листинг 7 – Запрос на получение данных таблицы warehouse с фильтрацией.

```
SELECT * FROM warehouse
WHERE shop_id = 1;
```

В листинге 7 показывается выполнение запроса с фильтрацией. Для этого используется ключевое слово WHERE, после чего указывается поле, по которому будут фильтровать данные, и значение, которое будет искать в строках таблицы.

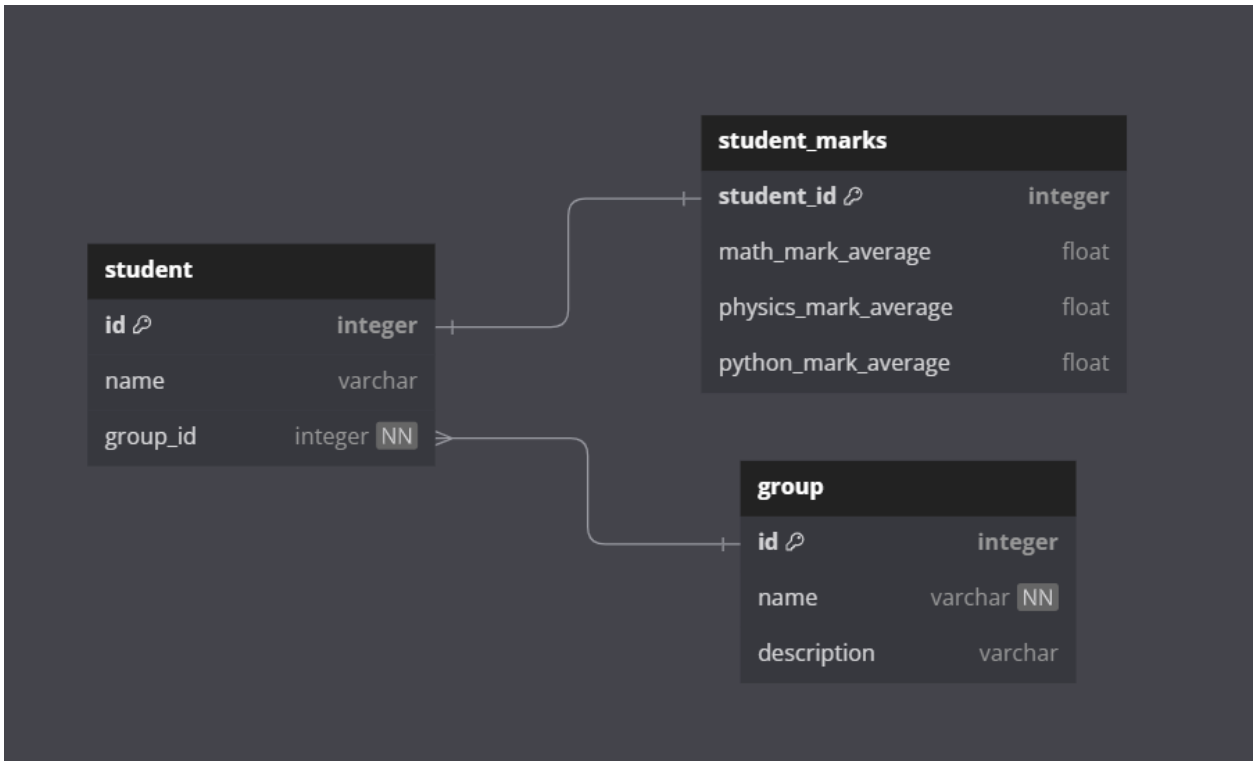
Задания

1. Создать таблицы на основе рисунка 2. Создание должно производится через скрипты.

2. Внесите в каждую таблицу данные. (Для каждой таблицы создайте не менее пяти записей, самостоятельно определив значения полей.)

3. Специально внести некорректные данные и исправить

4. К каждой таблице написать запросы с фильтрацией. Результаты фильтрации отобразить в виде скриншотов и скачать файл итоговой работы.



Общий код

```
CREATE TABLE study_group (
id INTEGER PRIMARY KEY,
name VARCHAR(255) NOT NULL,
description VARCHAR(255));

CREATE TABLE student (
id INTEGER PRIMARY KEY,
name VARCHAR(255),
group_id INTEGER REFERENCES study_group (id) NOT NULL);
```

```
CREATE TABLE student_marks (
student_id INTEGER REFERENCES student (id) PRIMARY KEY,
math_mark_average FLOAT,
physics_mark_average FLOAT,
python_mark_average FLOAT);

INSERT INTO study_group (id, name, description) VALUES (1, '11А', 'strong');
INSERT INTO study_group (id, name, description) VALUES (2, '11Б', 'creative');
INSERT INTO study_group (id, name, description) VALUES (3, '11В', 'smart');
INSERT INTO study_group (id, name, description) VALUES (4, '11Г', 'dramatic');
INSERT INTO study_group (id, name, description) VALUES (5, '11Д', 'olimp');


INSERT INTO student VALUES (1, 'Станислав', 1);
INSERT INTO student VALUES (2, 'Глеб', 1);
INSERT INTO student VALUES (3, 'Фома', 2);
INSERT INTO student VALUES (4, 'Юрий', 2);
INSERT INTO student VALUES (5, 'Игнат', 3);
INSERT INTO student VALUES (6, 'Наталья', 3);
INSERT INTO student VALUES (7, 'Роман', 4);
INSERT INTO student VALUES (8, 'Владислав', 4);
INSERT INTO student VALUES (9, 'Феофан', 5);
/// ЗДЕСЬ МОЖНО СДЕЛАТЬ ОШИБКУ (вписать id = 8, но id - unique)
INSERT INTO student VALUES (10, 'Виктор', 5);


INSERT INTO student_marks VALUES (1, 4.2, 4.8, 5.0);
INSERT INTO student_marks VALUES (2, 4.4, 4.1, 4.9);
```

INSERT INTO student_marks VALUES (3, 4.6, 3.8, 4.6);

INSERT INTO student_marks VALUES (4, 4.8, 3.7, 4.8);

INSERT INTO student_marks VALUES (5, 4.9, 3.9, 4.9);

INSERT INTO student_marks VALUES (6, 4.8, 4.6, 5.0);

INSERT INTO student_marks VALUES (7, 4.1, 4.7, 5.0);

INSERT INTO student_marks VALUES (8, 4.9, 4.9, 4.4);

INSERT INTO student_marks VALUES (9, 4.0, 3.6, 4.7);

INSERT INTO student_marks VALUES (10, 4.2, 3.6, 4.8);

SELECT * FROM study_group;

SELECT * FROM student

WHERE group_id <= 3;

SELECT * FROM student_marks

WHERE physics_mark_average >= 3.9;

Ход выполнения

Создание таблиц БД

```
CREATE TABLE study_group (
id INTEGER PRIMARY KEY,
name VARCHAR(255) NOT NULL,
description VARCHAR(255));
```

RUN SQL

Внесение данных

```sql
INSERT INTO study_group (id, name, description) VALUES (1, '11А', 'strong');
INSERT INTO study_group (id, name, description) VALUES (2, '11Б', 'creative');
INSERT INTO study_group (id, name, description) VALUES (3, '11В', 'smart');
INSERT INTO study_group (id, name, description) VALUES (4, '11Г', 'dramatic');
```

RUN SQL

```sql
INSERT INTO student VALUES (8, 'Владислав', 4);
INSERT INTO student VALUES (8, 'Феофан', 5);
INSERT INTO student VALUES (10, 'Виктор', 5);
```

RUN SQL

TEMPLATES

Error UNIQUE constraint failed: student.id

Исправив id, нужно добавить Феофана с id = 9 и Виктора (10)

```sql
INSERT INTO student_marks VALUES (8, 4.9, 4.9, 4.4);
INSERT INTO student_marks VALUES (9, 4.0, 3.6, 4.7);
INSERT INTO student_marks VALUES (10, 4.2, 3.6, 4.8);
```

RUN SQL

Создание запросов

```sql
SELECT * FROM study_group;
```

RUN SQL

id	name	description
1	11А	strong
2	11Б	creative
3	11В	smart
4	11Г	dramatic
5	11Д	olimp

```sql
SELECT * FROM student
WHERE group_id <= 3;
```

RUN SQL

id	name	group_id
1	Станислав	1
2	Глеб	1
3	Фома	2
4	Юрий	2
5	Игнат	3
6	Наталья	3

```sql
SELECT * FROM student_marks
WHERE physics_mark_average >= 3.9;
```

RUN SQL

student_id	math_mark_average	physics_mark_average	python_mark_average
1	4.2	4.8	5
2	4.4	4.1	4.9
5	4.9	3.9	4.9
6	4.8	4.6	5
7	4.1	4.7	5
8	4.9	4.9	4.4

Заключение

Получилось провзаимодействовать с базовыми возможностями SQL и попробовать выполнить основные возможные задачи.